1.Markdown

```
# Vehicle Tracking Backend
**Frontend Developer Integration Guide**
**Test Results, API Structure & Real-time Data**

**Version:** 1.2 (Updated with Geofences Endpoint)
**Date:** January 09, 2026
**Environment:** Test / Development only (do NOT use in production)

## 1. Environment & Access

- **Domain/Base URL:** `https://YOUR-TEST-DOMAIN.ddns.net` (replace with your actual test domain, e.g. caritest.ddns.net)
- **REST API Base:** `https://YOUR-TEST-DOMAIN.ddns.net/backend/api`
- **Socket.IO Base:** `https://YOUR-TEST-DOMAIN.ddns.net`
- **Socket.IO Path:** `/backend/socket.io` (must match exactly)

**Mandatory Header for all REST requests:**
```http
x-api-key: YOUR_API_KEY_HERE
```
```

2. Authentication Flow

POST /api/login – Get session token

- **Method:** POST
- **Headers:**
  - Content-Type: application/json
  - x-api-key: YOUR_API_KEY_HERE
- **Body (JSON):**

  JSON

  ```
  {
    "userid": "YOUR TEST USERID",
    "password": "YOUR TEST PASSWORD"
  }
  ```

- **Success Response (example):**

  JSON

  ```
  {
    "userid": "YOUR_TEST_USERID",
    "accountid": YOUR ACCOUNT ID,
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
  ```

- **Error:** 401/400 + message if fails (check res.ok)

Use the returned **token** + **x-api-key** for Socket.IO auth (see section 4).

Security Reminder: Never hardcode credentials. Use .env files during development.

3. REST Endpoints – Tested Examples

Get Vehicle Groups

- **Endpoint:**GET /groups
- **Query Params:**?accountId=YOUR_ACCOUNT_ID (required)
- **Headers:**x-api-key: YOUR_API_KEY_HERE
- **Example curl:**

  Bash

  ```
  curl -H "x-api-key: YOUR_API_KEY_HERE" \
  "https://YOUR-TEST-
  DOMAIN.ddns.net/backend/api/groups?accountId=YOUR_ACCOUNT_ID"
  ```

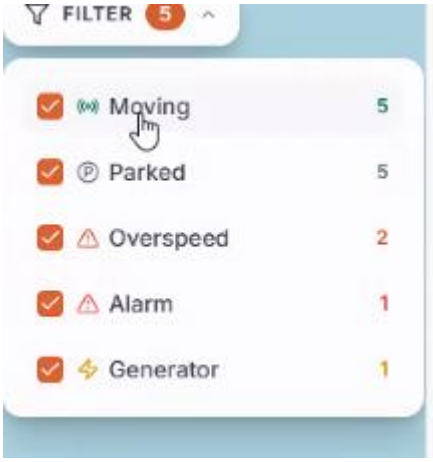
- **Response:** Array of group objects

  JSON


  ```
  [
    {
      "object_group_id": 1,
      "object_group_name": "Pomp Truck",
      "object_group_image": null,
      "total": 5
    },
    {
      "object_group_id": 2,
      "object_group_name": "Beton Truck",
      "object_group_image": null,
      "total": 17
    },
    {
      "object group id": 3,
      "object group name": "Trekker",
      "object_group_image": null,
      "total": 4
    },
    {
      "object_group_id": 4,
      "object_group_name": "Kraan Truck",
  ```


```
    "object_group_image": null,
    "total": 1
  },
  {
    "object_group_id": 5,
    "object_group_name": "Dump Truck",
    "object_group_image": null,
    "total": 1
  }
]
```


Information for component



Get Geofences (New!)

- **Endpoint:**GET /geofences
- **Query Params:**?accountId=YOUR_ACCOUNT_ID (required)
- **Headers:**x-api-key: YOUR_API_KEY_HERE
- **Example curl:**

  Bash

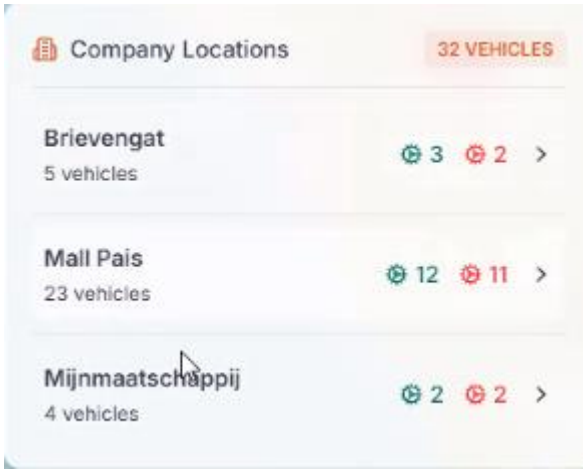
  ```
  curl -H "x-api-key: YOUR_API_KEY_HERE" \
  "https://YOUR-TEST-
  DOMAIN.ddns.net/backend/api/geofences?accountId=YOUR_ACCOUNT_ID"
  ```

- **Response:** Array of geofence objects

  JSON

  ```
  [
    {
      "geofence_id": 47,
      "geofence_name": "BIB",
      "vehicle_count": 23
    },
    {
      "geofence_id": 48,
      "geofence_name": "MAL PAIS",
      "vehicle_count": 0
    },
    {
      "geofence_id": 49,
      "geofence_name": "MMC",
      "vehicle_count": 4
    }
  ]
  ```

Information for component



4. Real-time Live Data via Socket.IO

Backend pushes updates **every ~5 seconds** after successful connection.

Key Connection Details

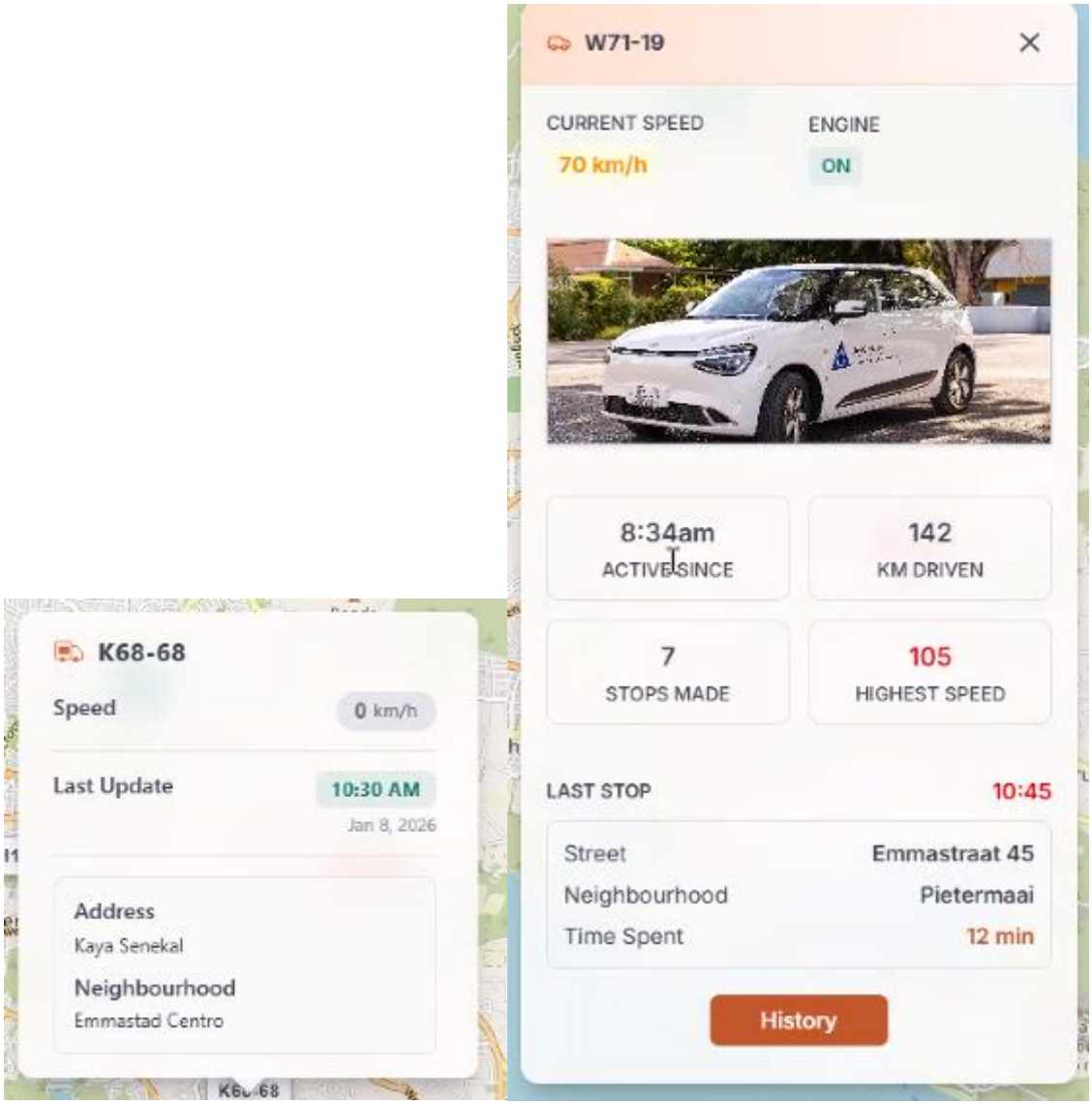
- Library: socket.io-client
- Auth object: { apiKey: "YOUR_API_KEY_HERE", token: "session.token" }
- Path: /backend/socket.io
- Recommended transports: ["websocket"]

Vehicle Object Schema (from vehicles:update)

TypeScript

```
interface Vehicle {
  object_id: number;
  label: string;                        // e.g. "508"
  position: [number, number];           // [lat, lng]
  time: string;                         // e.g. "05:45 PM"
  date: string;                         // e.g. "Jan 08, 2026"
  speed: number;
  heading: number;
  street: string;
  neighborhood: string;
  engine: "ON" | "OFF";
  KMdriven: number;
  stopmade: number;
  last_stop_datetime: string;
  last_stop_street: string;
  last_stop_neighborhood: string;
  last_stop_time_spent: string;         // "NA" or duration
  group_id: number;
}
```

used in component

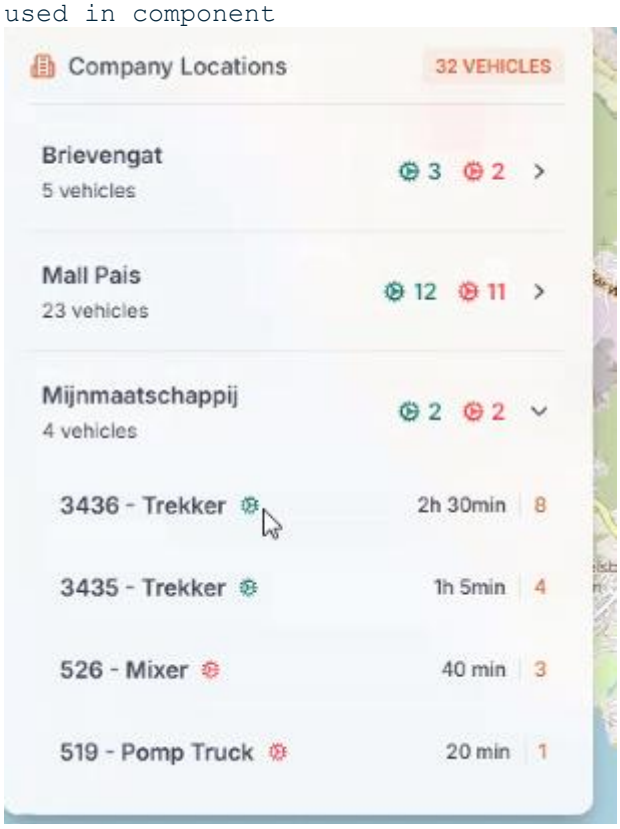


Geofence Vehicle Object Schema (from geofence:vehicles:update)

TypeScript

```typescript
interface VehicleInGeofence {
  EngineStatus: boolean;              // true = ON, false = OFF
  deviceid: number;
  geofenceId: number;
  geofenceName: string;               // e.g. "BIB"
  label: string;
  gpsDate: string;                    // ISO timestamp
  gfArriveTime: string;               // Arrival ISO timestamp
  vehicleModel: string;               // e.g. "Beton Truck"
  today_total_trips: number;
  time_inside: string;                // e.g. "44h 23min"
}
```

used in component

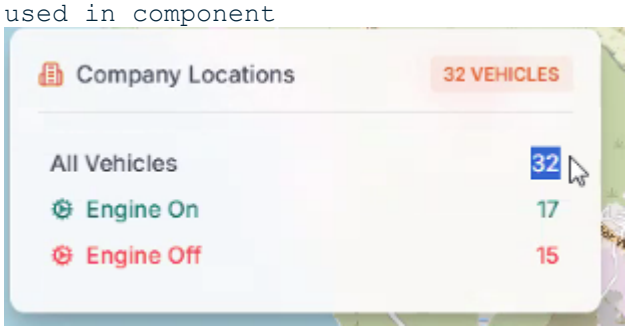


Dashboard Summary Schema

TypeScript

```
interface DashboardSummary {
  totalVehicles: number;
  engineOn: number;
  engineOff: number;
}
```

used in component



Emitted Events

- vehicles:update → Array<Vehicle> (full list, ~28 vehicles in test)
- geofence:vehicles:update → { [geofenceId: string]: Array<VehicleInGeofence> }
- dashboard:summary → DashboardSummary

Test observation: ~28 vehicles total, 1 engine ON / 27 OFF, multiple geofences (e.g. BIB with many vehicles).

5. Recommended Test Script (Node.js)

JavaScript

```
// socket_test.js
const { io } = require("socket.io-client");

const BACKEND = "https://YOUR-TEST-DOMAIN.ddns.net";
const API_BASE = `${BACKEND}/backend/api`;
const API_KEY = "YOUR_API_KEY_HERE";

async function login() {
  const res = await fetch(`${API_BASE}/login`, {
    method: "POST",
    headers: { "Content-Type": "application/json", "x-api-key": API_KEY },
    body: JSON.stringify({ userid: "YOUR_TEST_USERID", password:
"YOUR_TEST_PASSWORD" })
  });
  if (!res.ok) throw new Error(`Login failed: ${res.status} - ${await
res.text()}`);
  return await res.json();
}

async function main() {
  try {
    const session = await login();
    console.log("✅ Logged in! User:", session.userid, "| Account:",
session.accountid);

    const socket = io(BACKEND, {
      path: "/backend/socket.io",
      transports: ["websocket"],
      auth: { apiKey: API_KEY, token: session.token }
    });

    socket.on("connect", () => console.log("✅ Socket connected! ID:",
socket.id));
    socket.on("connect_error", err => console.error("❌ Connect error:",
err.message));
```

```
    socket.on("disconnect", reason => console.log("Disconnected:",
reason));

    socket.on("vehicles:update", vehicles => {
      console.log(`\n🚗 ${vehicles.length} vehicles received`);
      vehicles.slice(0, 3).forEach(v => console.log(v));
      if (vehicles.length > 3) console.log(`   ... +${vehicles.length - 3}
more`);
    });

    socket.on("geofence:vehicles:update", groups => {
      const count = Object.keys(groups).length;
      console.log(`\n📍 ${count} geofence(s) with vehicles`);
      Object.entries(groups).forEach(([id, vehs]) => {
        console.log(`   Geofence ${id} (${vehs[0]?.geofenceName || '?'}) :
${vehs.length} vehs`);
      });
    });

    socket.on("dashboard:summary", s => {
      console.log("\n═══════════════════════════════════════════════════");
      console.log("          Company Locations");
      console.log("═══════════════════════════════════════════════════");
      console.log(`   All Vehicles           ${s.totalVehicles} VEHICLES`);
      console.log(`   Engine On              ${s.engineOn}`);
      console.log(`   Engine Off             ${s.engineOff}`);
      console.log("═══════════════════════════════════════════════════");
    });

    console.log("\n Listening... Press Ctrl+C to stop\n");
  } catch (err) {
    console.error("❌ Error:", err.message);
  }
}

main();
```

Run: node socket_test.js

6. Best Practices & Security

- Always include x-api-key on REST calls
- Socket.IO requires **both**apiKey and token in auth
- Use exact Socket path: /backend/socket.io
- Store all credentials in **environment variables** (never commit)
- Add **reconnect logic** (reconnection: true)
- Handle token expiration → force re-login
- After testing: **Immediately rotate API key** and change test password
- For production: Use dedicated keys, short-lived tokens, rate limiting

Contact [your name/email] for more endpoints, production setup, or questions.

Happy integrating!